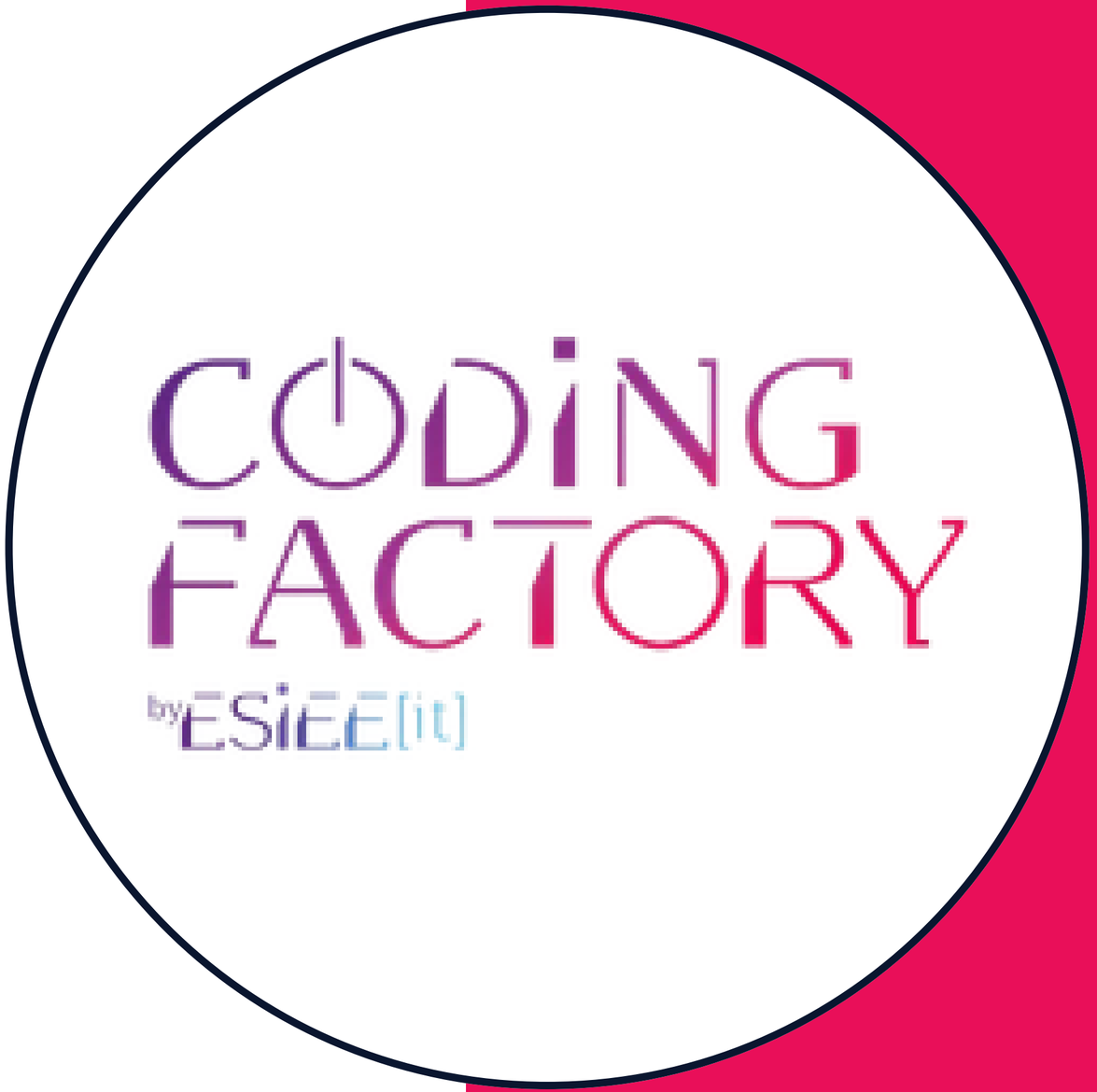
Les 5 6 et 9 Mars 2026

# Formation Typescript M2i Lead Dev M1 BNP




Francesco HARTEMANN

# Ressources

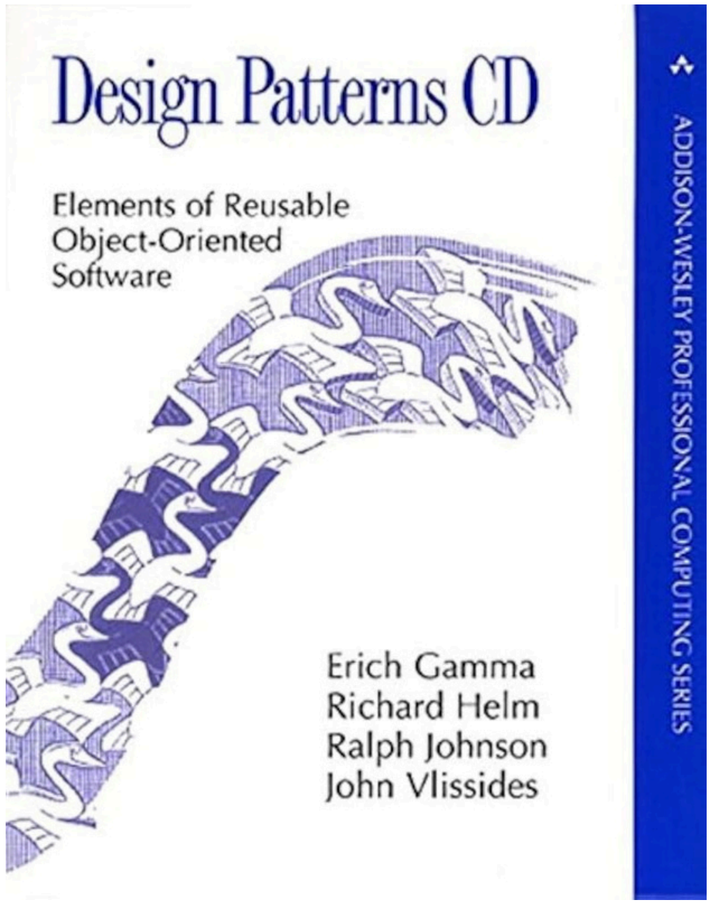


- Slides et notes de cours
- Typescript : https://git-scm.com/doc
- NodeJS : https://www.typescriptlang.org/
- NestJS : https://nodejs.org/en
- Prisma : Pour des questions spécifiques

# 03

# DESIGN PATTERNS

wom


Design Patterns CD
Elements of Reusable
Object-Oriented
Software
Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides
ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES

1998

O'REILLY
Second Edition
Head First
Design Patterns
Building Extensible & Maintainable Object-Oriented Software
Eric Freeman & Elisabeth Robson
with Kathy Sierra & Bert Bates
A Brain-Friendly Guide

1998

**The Catalog of Design Patterns**

The catalog of design patterns grouped by intent, complexity, and popularity. The catalog contains all classic design patterns and several architectural patterns.

refactoring.guru

07

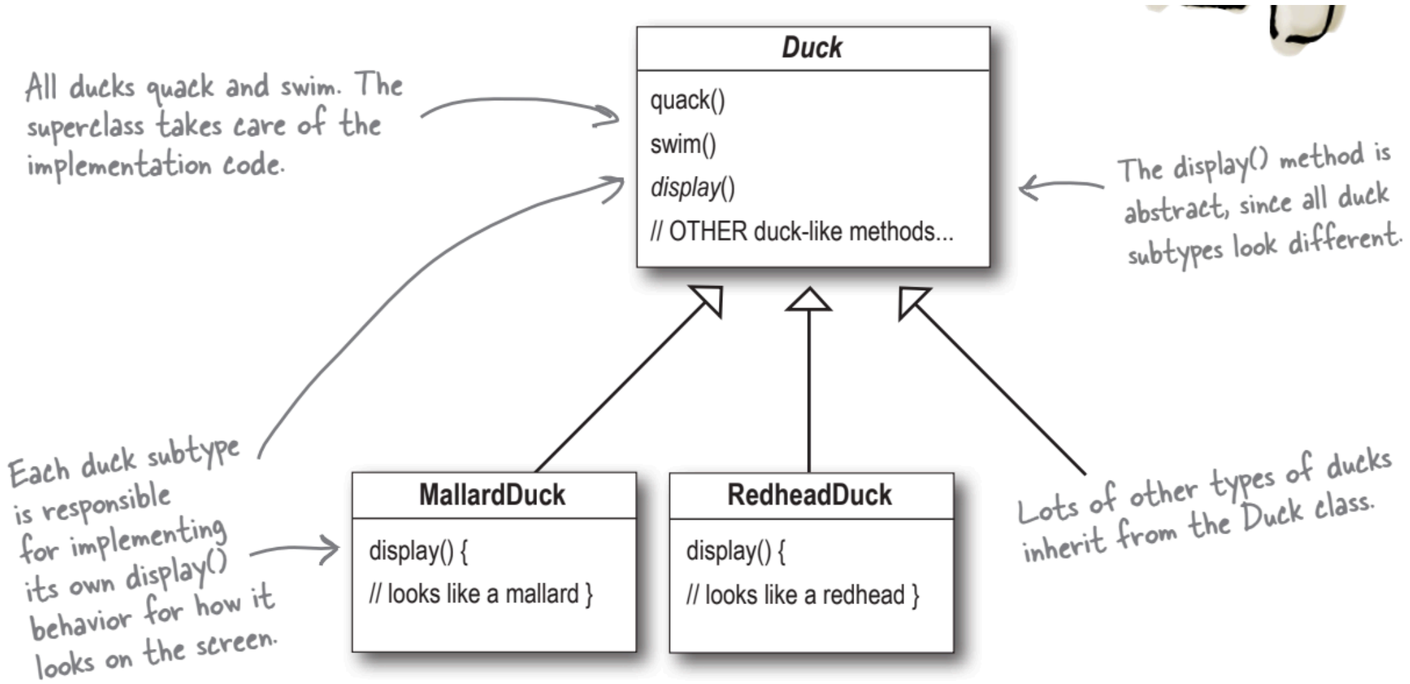
# STRATEGY

# 8

## STRATEGY

**Le pattern Strategy** définit une famille d’algorithmes, encapsule chacun d’eux et les rend interchangeables.
Il permet à l’algorithme de varier indépendamment des clients qui l’utilisent.

# DUCK - STRATEGY



10

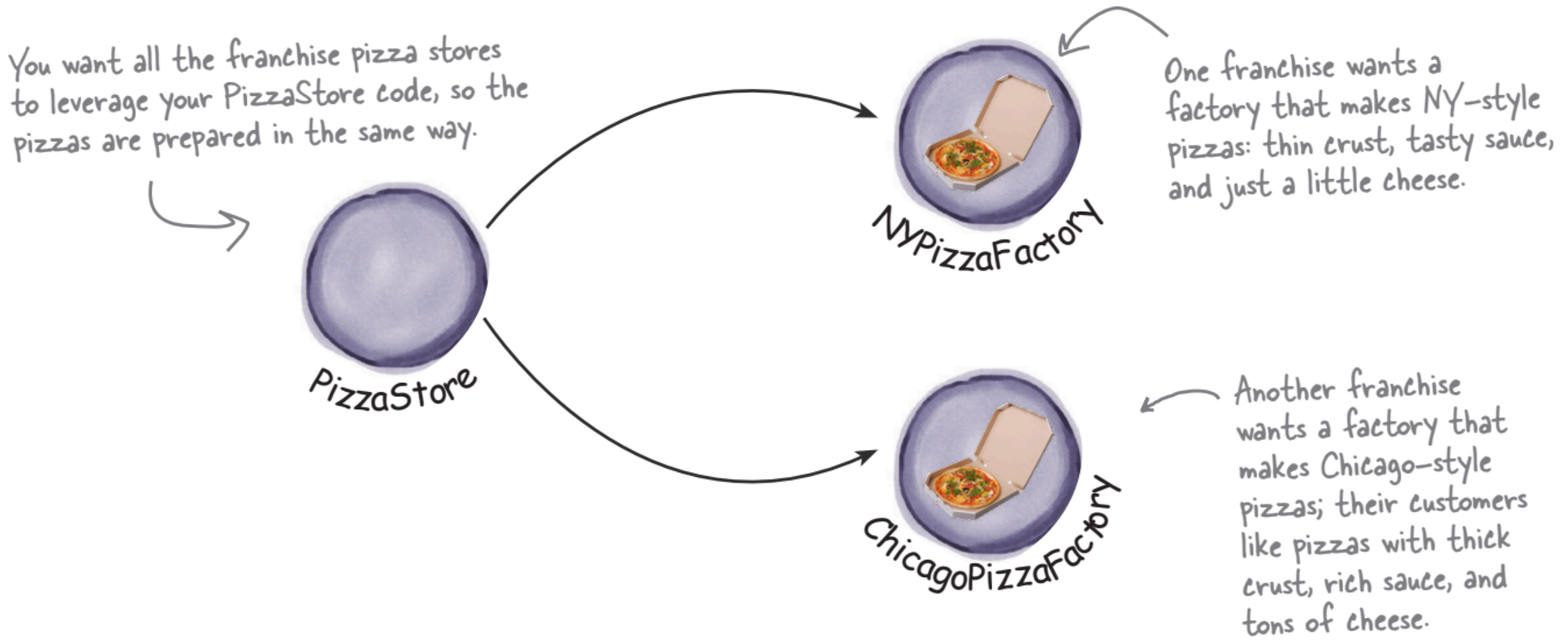
# FACTORY

# SIMPLE PIZZA FACTORY

Tous les patterns de type Factory encapsulent la création d’objets.
Le Factory Method Pattern encapsule la création d’objets en laissant aux sous-classes le soin de décider quels objets créer.

# 12

# FRANCHISE PIZZA - FACTORY



13

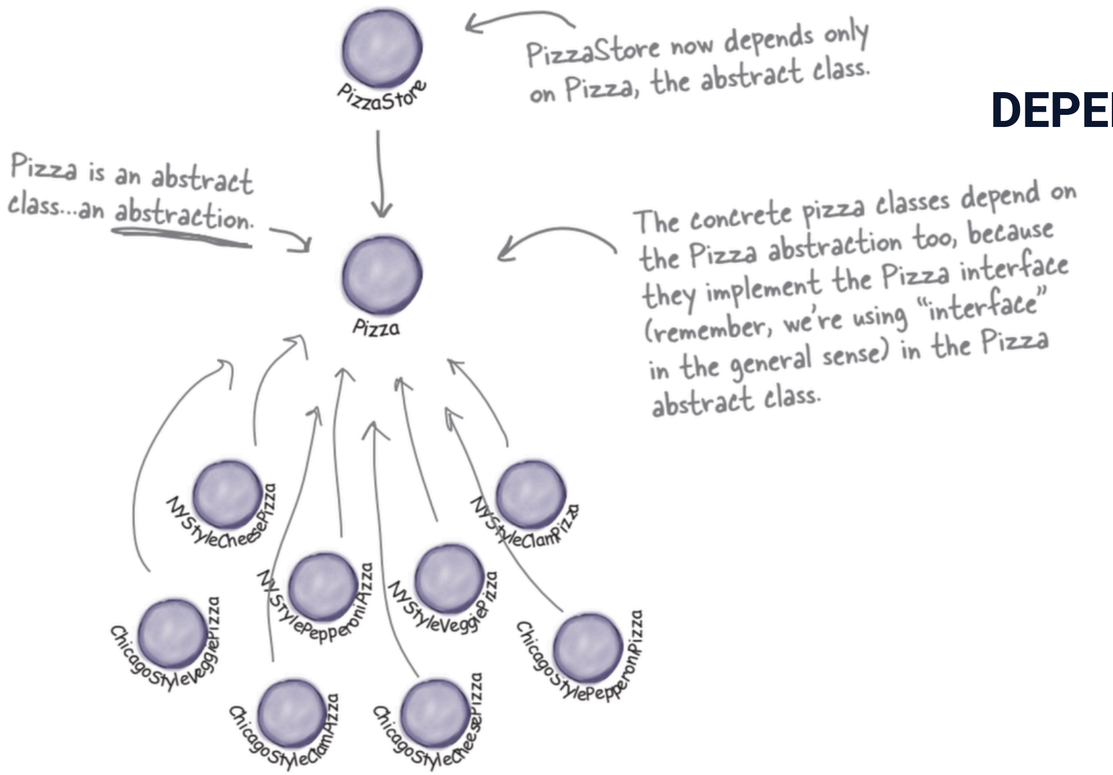
# ABSTRACT FACTORY

# ABSTRACT FACTORY

Abstract Factory fournit une interface permettant de créer des familles d’objets liés ou interdépendants, sans avoir à spécifier leurs classes concrètes.

When you directly instantiate an object, you are depending on its concrete class.

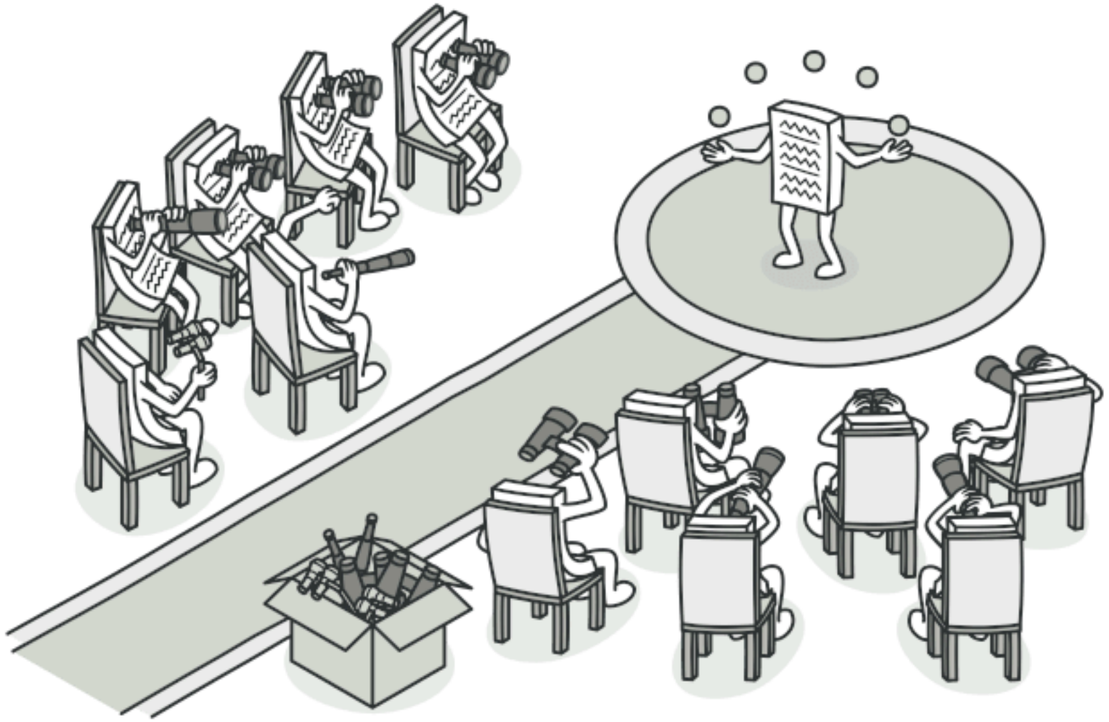
# DEPENDENCY INVERSION PRINCIPLE.



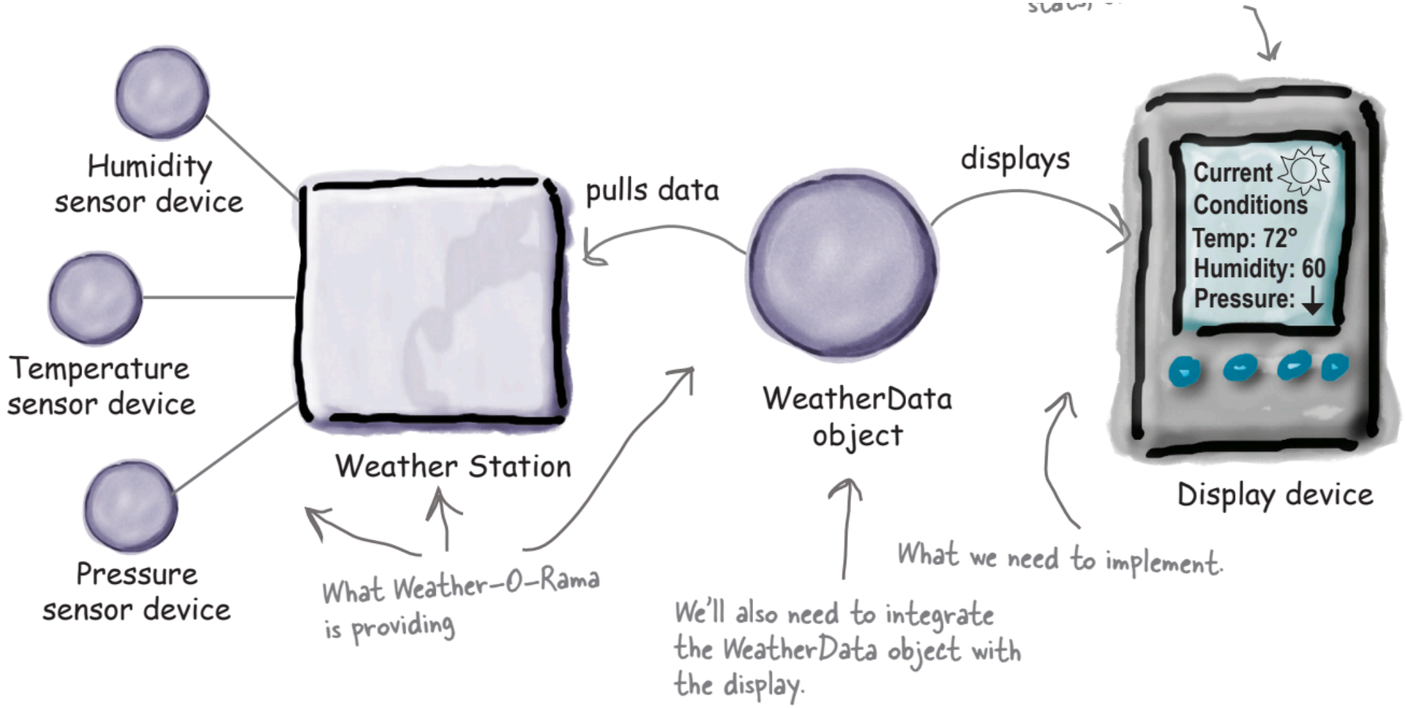
17

# OBSERVER

# OBSERVER

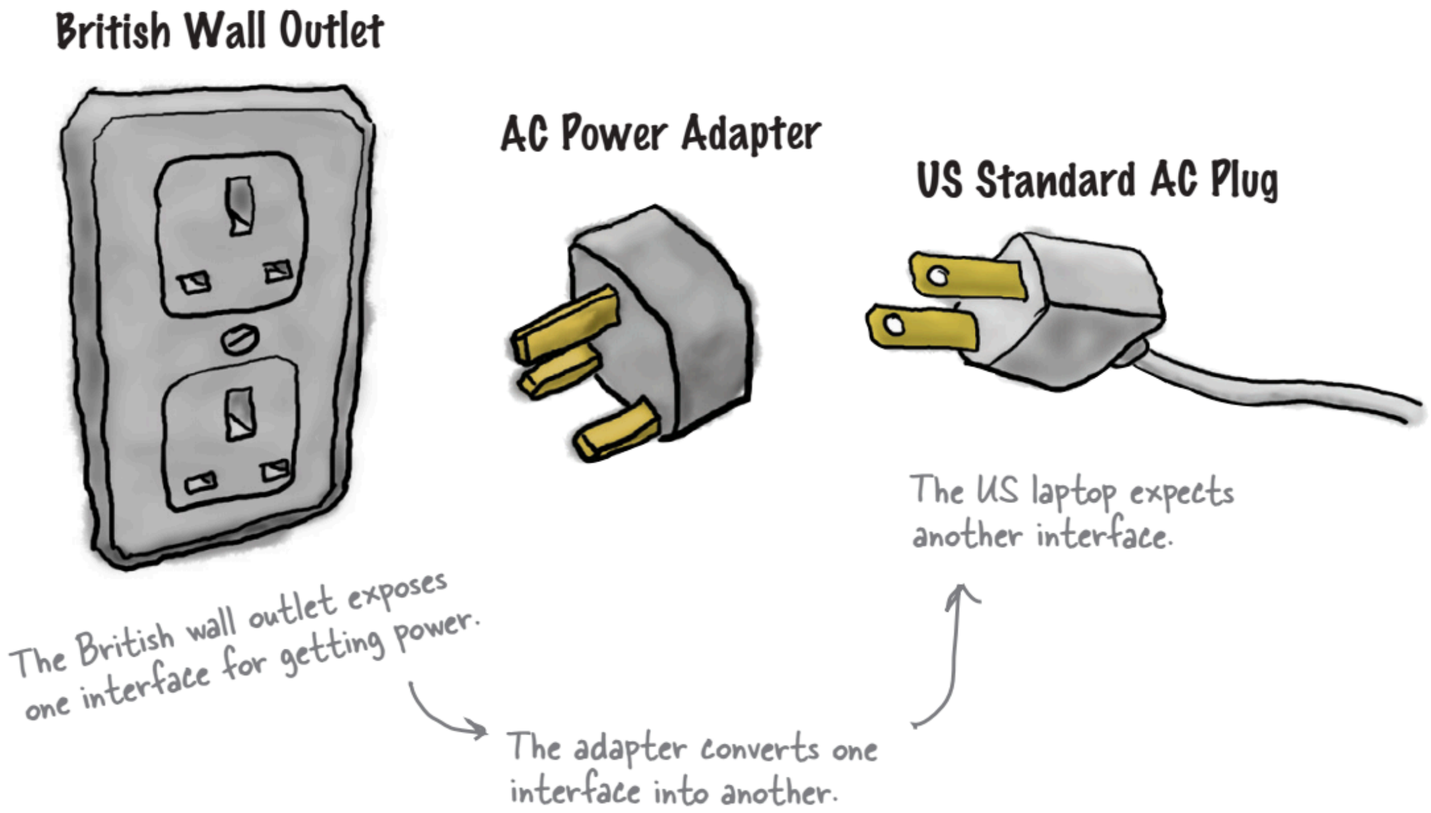


Le pattern Observer définit une relation un-à-plusieurs entre des objets, de sorte que lorsqu'un objet change d'état, tous ses dépendants sont automatiquement notifiés et mis à jour.

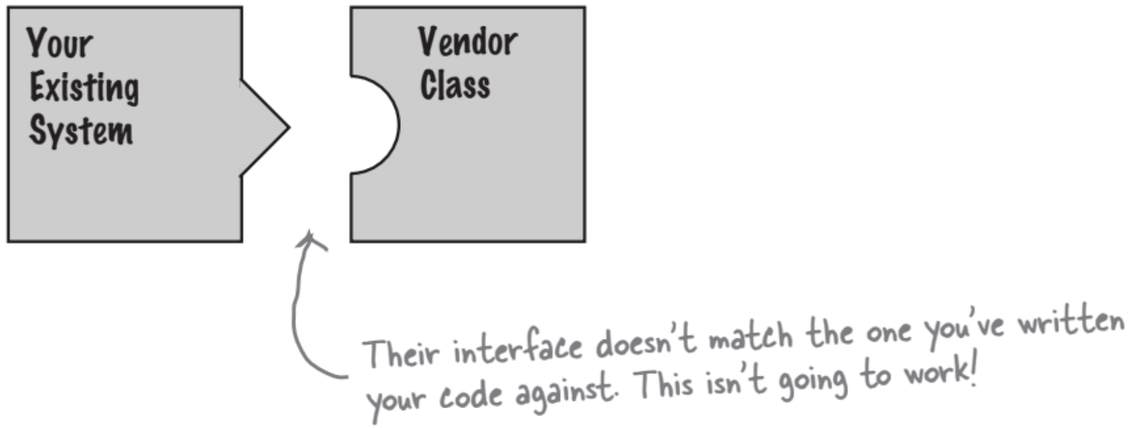

Humidity sensor device
Temperature sensor device
Pressure sensor device
Weather Station
pulls data
WeatherData object
displays
Current Conditions
Temp: 72°
Humidity: 60
Pressure:
Display device
What Weather-O-Rama is providing
We'll also need to integrate the WeatherData object with the display.
What we need to implement.

20

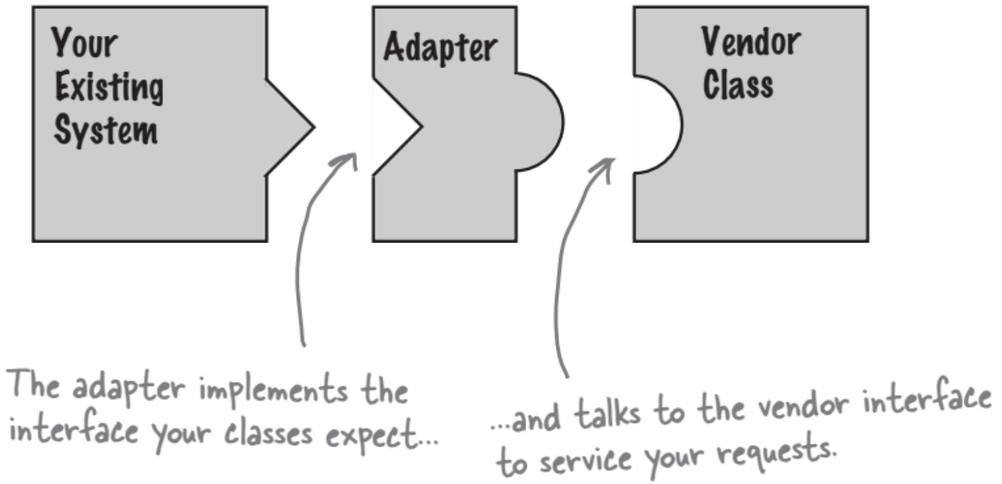
# ADAPTER


British Wall Outlet
AC Power Adapter
US Standard AC Plug
The US laptop expects another interface.
The British wall outlet exposes one interface for getting power.
The adapter converts one interface into another.

## PROBLEME


Your Existing System
Vendor Class
Their interface doesn't match the one you've written your code against. This isn't going to work!

## SOLUTION




Le pattern Adapter permet de faire collaborer deux interfaces incompatibles en introduisant une classe intermédiaire qui traduit l'interface existante vers celle attendue par le client, sans modifier le code source des classes concernées.

# SOLID

# L'ORIGINE DES PRINCIPES SOLID

Robert C. Martin

- Découvert par Robert C Martin et popularisé en 2004
- Principes visant à améliorer le design de code OO
- Très important pour le développement Agile

# QU'EST-CE QUE LE DESIGN ?

**Ingénierie**

Documents conçus par les ingénieurs passés à une autre équipe qui va construire le produit

**Informatique**

Le code source lui-même est ce document passé à un compilateur qui va construire le produit

EN RÉSUMÉ
Design = Code Source
Soigner le Design = Garantir la longévité du produit
SOLID aide à lutter contre les Design Smells

# ENTROPIE LOGICIELLE

Mauvais Design = Code difficile à changer

Or, un **Software** doit-être facile à changer !

## DESIGN SMELLS

30

# SINGLE RESPONSIBILITY PRINCIPLE

## DEFINITION

- Un module doit avoir une & une seule raison de changer
- Rassembler ce qui change pour les mêmes raisons

# OPEN CLOSED PRINCIPLE

## DEFINITION

- Un module doit-être ouvert à l'extension...
- Mais fermé à la modification
- Changer le comportement sans altérer le code source

# LISKOV SUBSTITUTION PRINCIPLE

## DEFINITION

- Les sous-types doivent être substituables à leur types parent
- Respect de l'héritage
- Interface = Type
  Implémentation = Sous-Type

# REFUSED BEQUEST

```
class User {
  login() {}
  sendNewPassword() {}
  delete() {}
}

class Admin extends User {
  delete() {
    throw new Error("An admin can't be deleted")
  }
}
```

- Le Sous-Type ré-écrit OU supprime une fonctionnalité hérité de son parent
- **Syndrôme de LSP**

# INTERFACE SEGREGATION PRINCIPLE

## DEFINITION

- Les clients d'un module ne doivent pas dépendre de ce qu'ils n'utilisent pas
- Dépendre de ce qu'on utilise

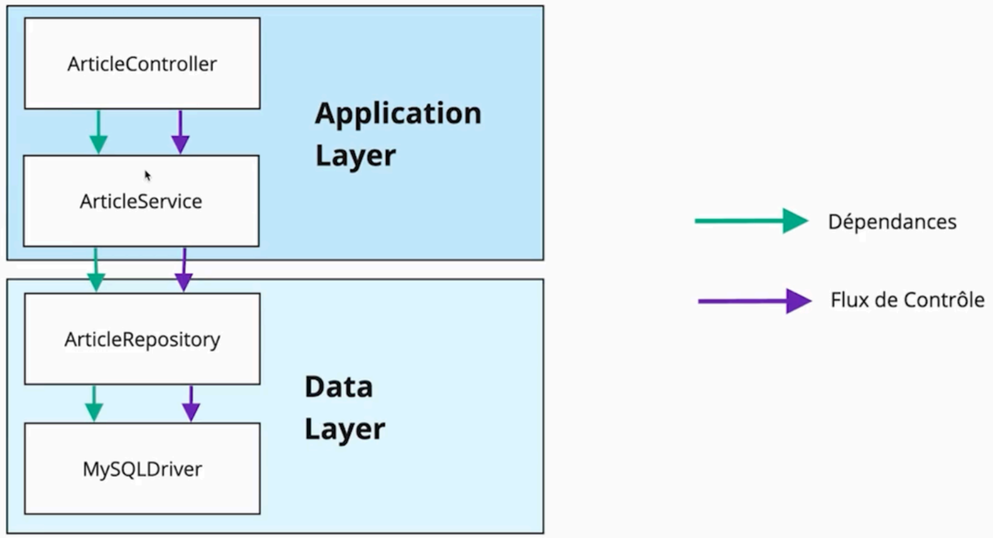
# DEPENDENCY INVERSION PRINCIPLE

## DEFINITION

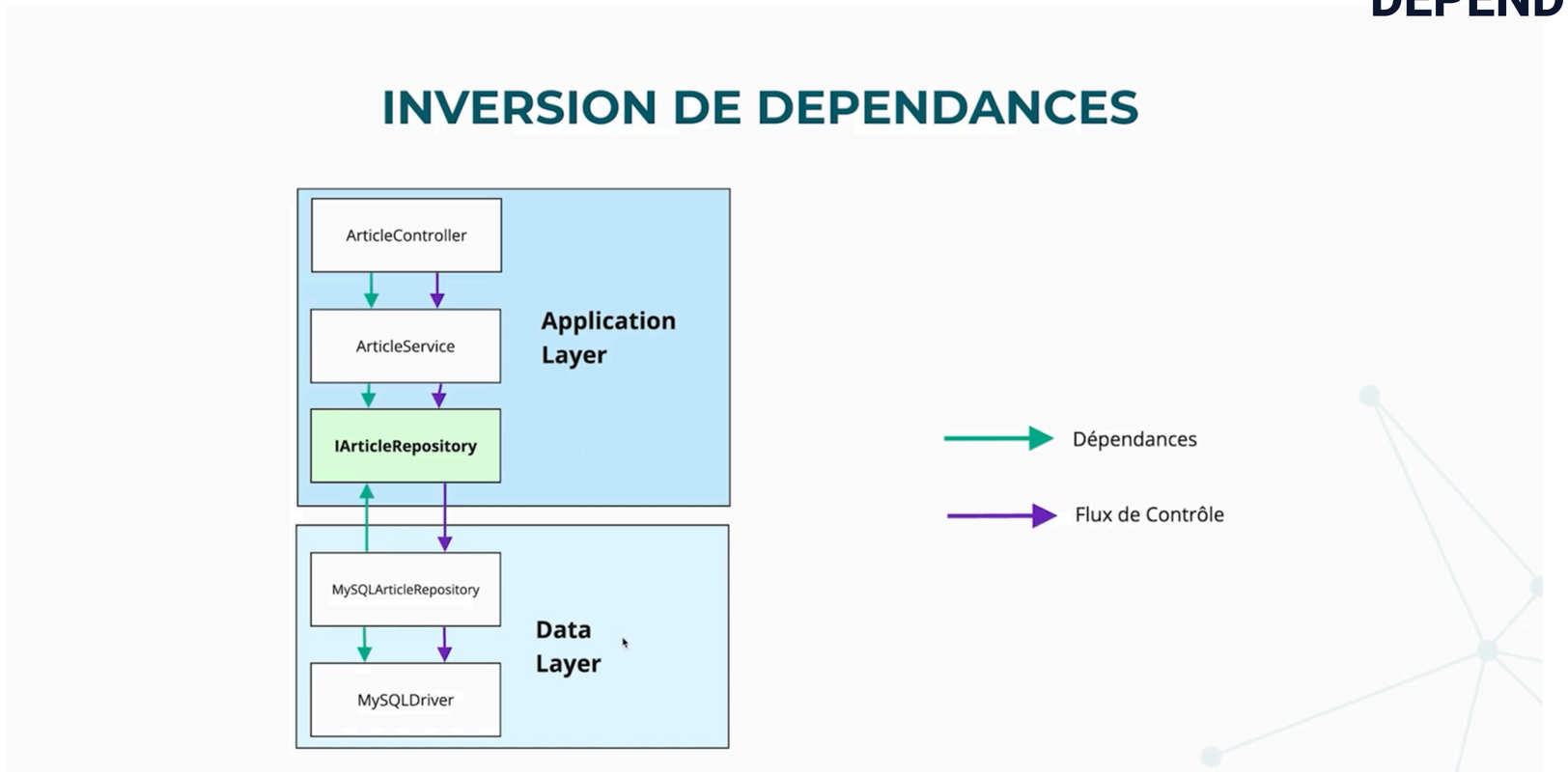
- Les abstractions ne doivent pas dépendre des détails
- Les détails doivent dépendre des abstractions

# ARCHITECTURE CONVENTIONNELLE

## STRUCTURE CONVENTIONNELLE



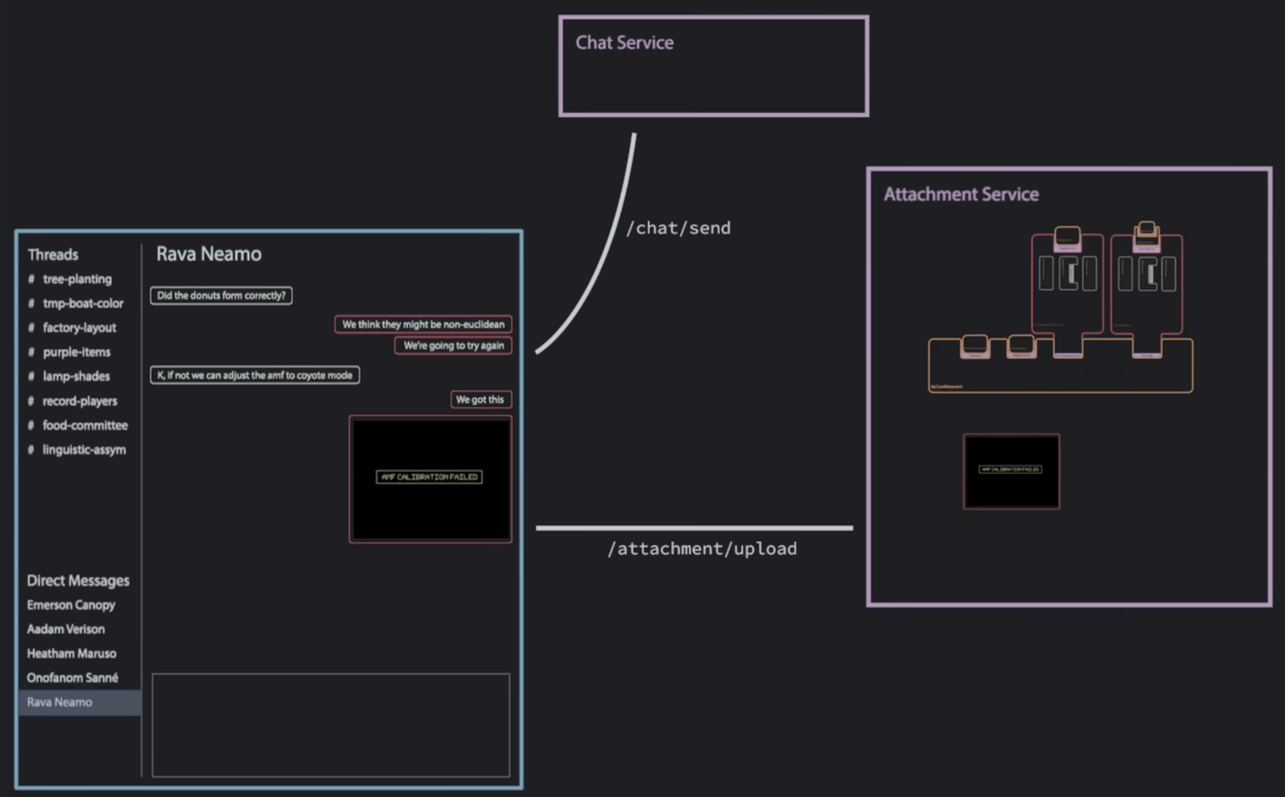
# DEPENDENCY INVERSION PRINCIPLE



# DEPENDECY INVERSION

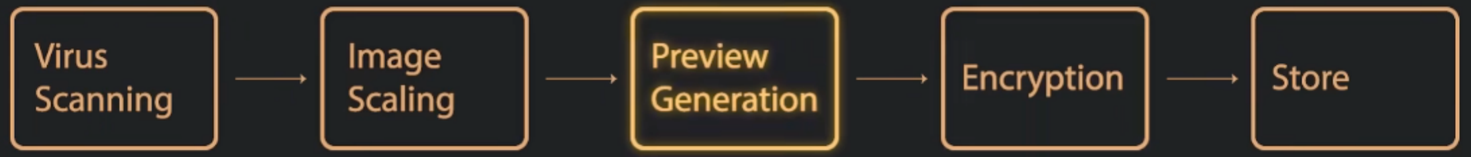
# 39

# CHAT APPLICATION

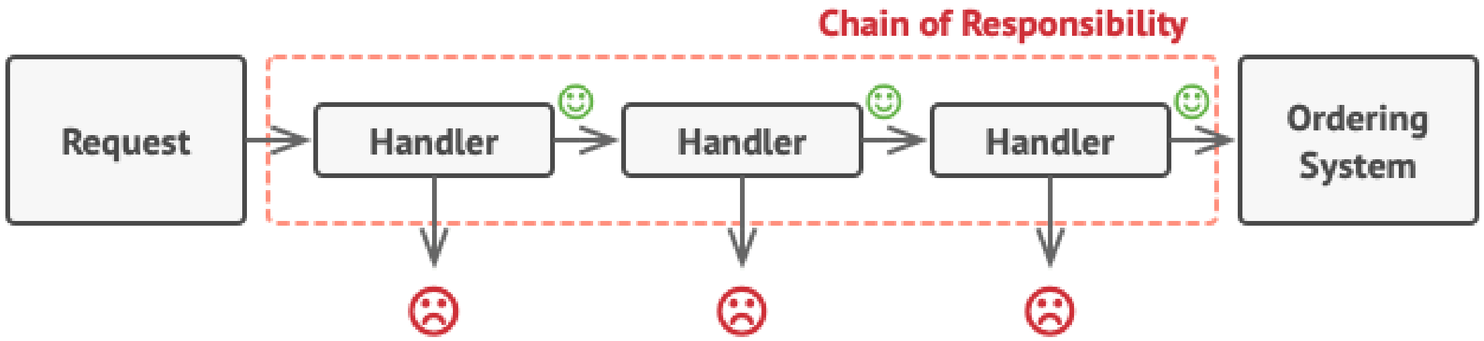


# 40

## UPLOAD MIDDLEWARE FLOW



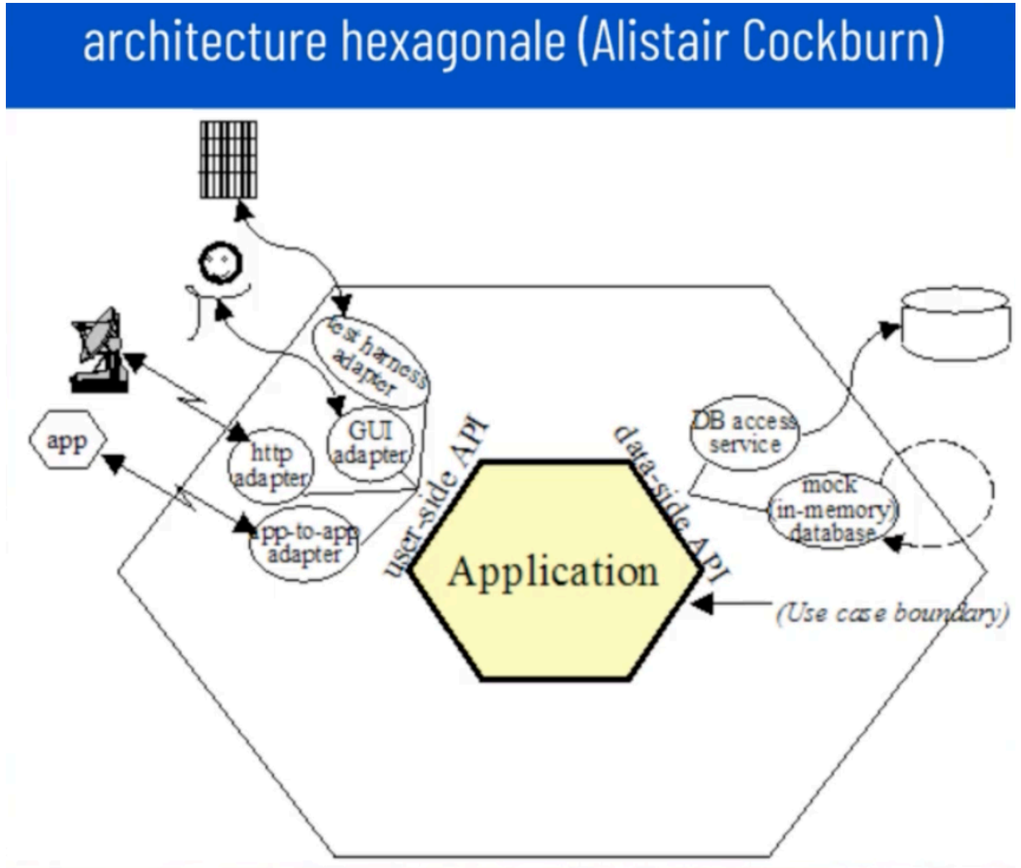
# CHAIN OF RESPONSSIBILITY




Le Chain of Responsibility est un design pattern comportemental qui permet de faire circuler une requête le long d'une chaîne de gestionnaires.
Lorsqu'un gestionnaire reçoit une requête, il décide soit de la traiter, soit de la transmettre au gestionnaire suivant dans la chaîne.

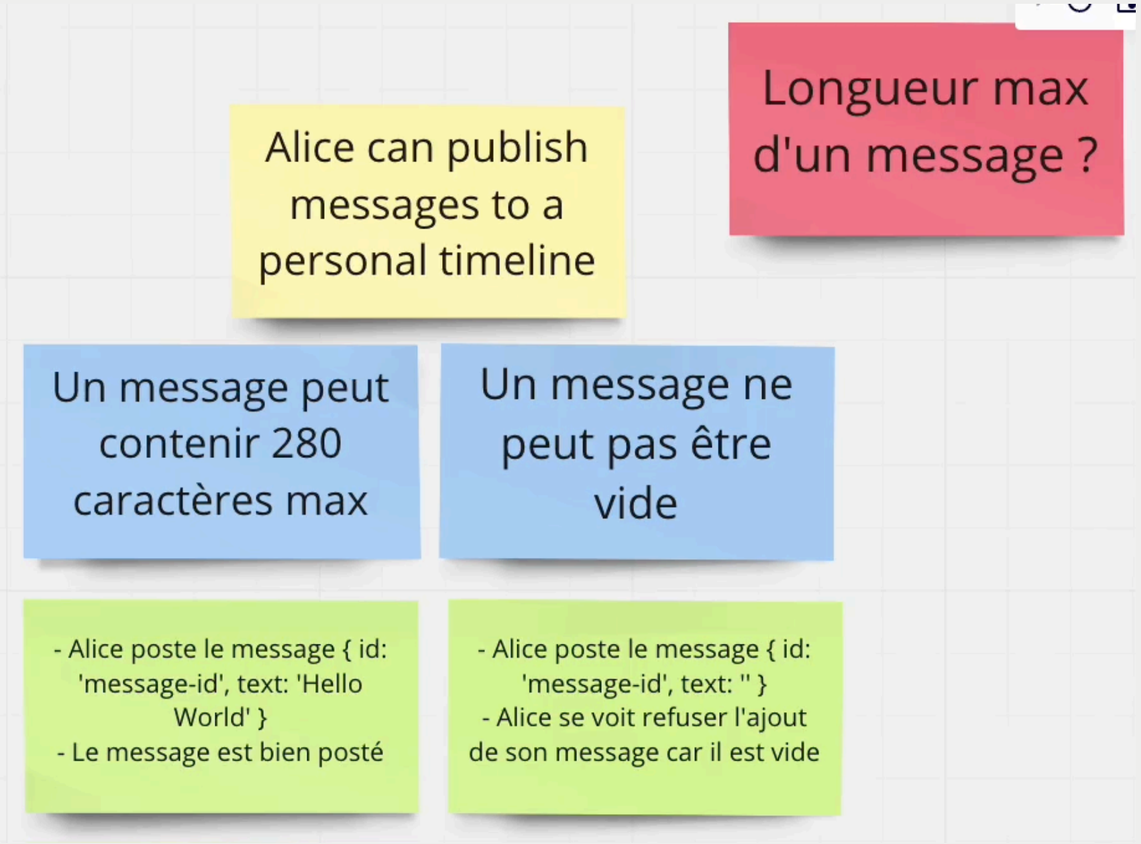
# HEXAGONAL ARCHITECTURE

## ORCHESTRATOR

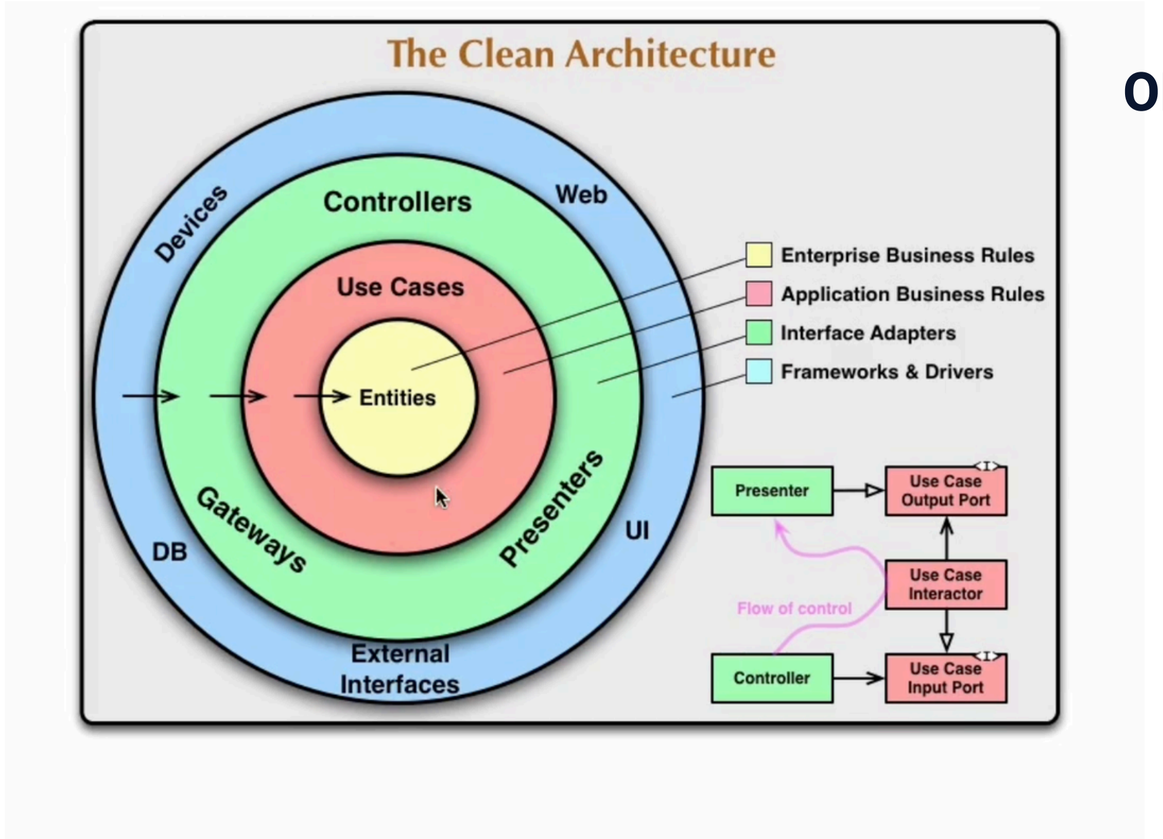


# SOCIAL NETWORKING KATA

# EXEMPLE MAPPING

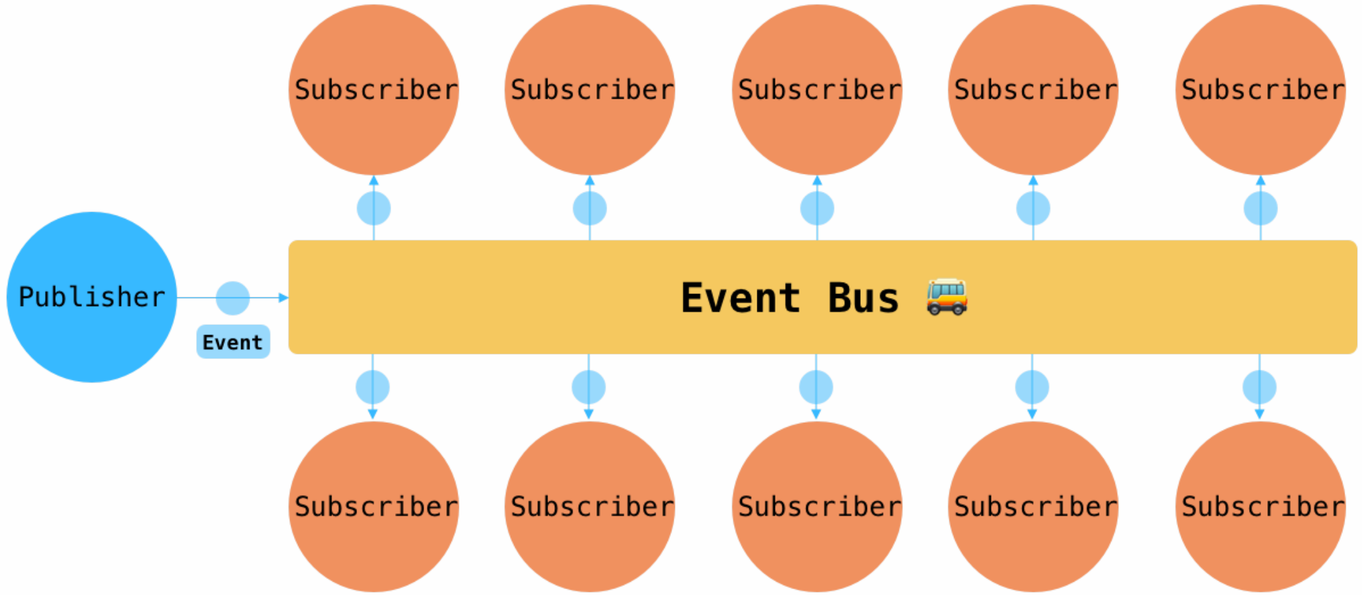


# ORCHESTRATOR - USECASE



# DDD – EVENT PROGRAMMATION

# EVENT BUS




Un bus d'événements est un pattern d'architecture logicielle qui facilite la communication entre différents composants ou modules d'un système, généralement en utilisant un modèle publication–abonnement (publish–subscribe).

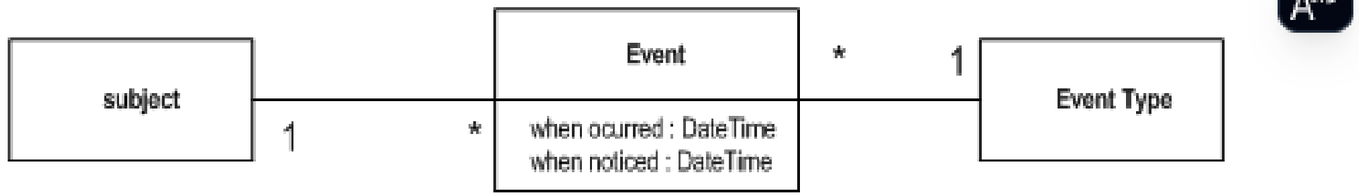
# AGGREGATE

L’agrégat est un pattern du Domain-Driven Design (DDD).
Un agrégat DDD est un ensemble d’objets du domaine qui peuvent être traités comme une seule unité.

Par exemple, une commande et ses lignes de commande sont des objets distincts, mais il est utile de traiter la commande (avec l’ensemble de ses lignes) comme un agrégat unique.

# Domain Event

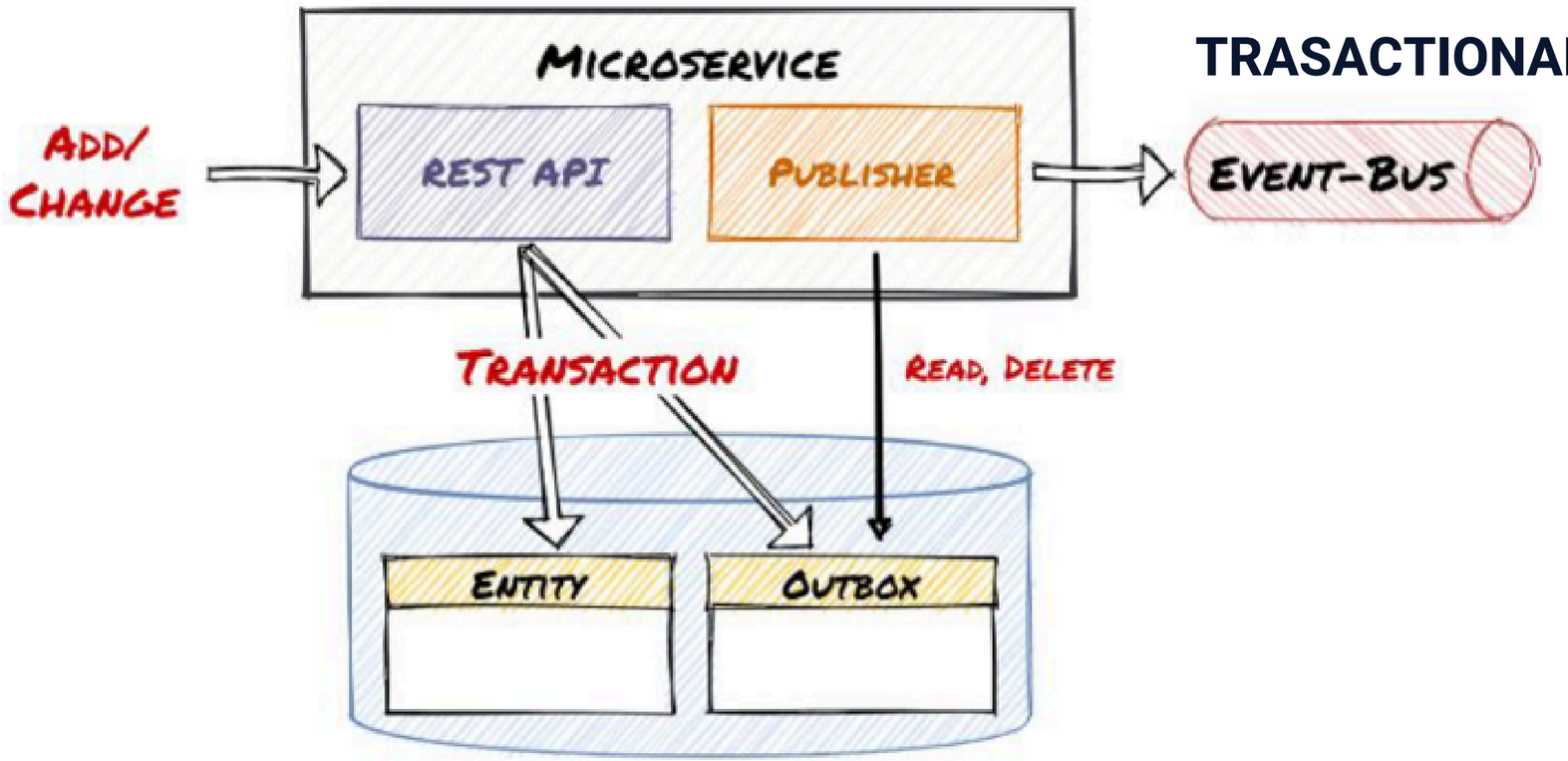
*Captures the memory of something interesting which affects the domain*




Capture la mémoire d’un événement ou d’un fait intéressant qui impacte le domaine.

Le Transactional Outbox Pattern est un pattern de fiabilité utilisé dans les architectures orientées événements pour garantir la cohérence des données entre une base de données et un broker de messages (Kafka, RabbitMQ, SNS/SQS, etc.).

## TRASACTIONAL OUTBOX PATTER



# RESOURCES

- Guru : https://refactoring.guru/
- Design Patterns: Elements of Reusable Object-Oriented Software  de Erich Gamma (Auteur), Richard Helm (Auteur), Ralph Johnson (Auteur), John Vlissides (Auteur)
- Head First Design Patterns de Elisabeth Freeman (Auteur), Eric Freeman (Auteur), Bert Bates (Auteur), Kathy Sierra (Auteur)
- Formation code durable: https://ancyracademy.fr/
- Formation clean archi : https://www.craftacademy.fr/
- Kata : https://codingdojo.org/
- Expert Architecture : https://www.martinfowler.com/
- Clean Architecture: A Craftsman's Guide to Software Structure and Design Livre de Robert C. Martin

# PROJET

## DESCRIPTION

Développer une API de commande de pizzas pour plusieurs enseignes.

Chaque enseigne a :
- Ses propres pizzas
- Ses propres prix
- Ses propres règles de préparation

Une marketplace veut regrouper plusieurs enseignes de pizza sur une seule plateforme.

Le client choisit une enseigne, compose sa commande, paie, et suit sa livraison.

| Enseigne | Exemple pizza | Règle de prix |
| --- | --- | --- |
| Da Michele | Margherita 9€ | S: -2€, L: +3€, XL: +5€ |
| Rossopomodoro | Supreme 12€ | S: ×0.8, L: ×1.3, XL: ×1.5 |
| Pizzium | Napolitaine 10€ | Taille unique M |

Chaque enseigne calcule ses prix différemment.

# FLOW D’UNE COMMANDE

Panier → Validation → Paiement → Suivi

✅ Important: Patterns, architecture, code propre

⚪ Secondaire: Auth, BDD, tests

# RENDU

📣 Un repo GitHub public par groupe devra être créé et son lien envoyé avant dimanche à 20 h, dernier délai !

60

CRITÈRES

- Patterns utilisés et justifiés
- Code découplé

# BACKLOG

# Story 1

62

ENSEIGNES ET PIZZAS

**En tant que système,**
**Je souhaite gérer plusieurs enseignes avec leurs pizzas**
**De sorte à proposer un catalogue varié**

Acceptance

- 3 enseignes : Da Michele, Rossopomodoro, Pizzium
- Chaque enseigne a son menu et ses règles de prix
- Ajouter une enseigne = ajouter une classe

wom

# Story 2

63

CALCUL DU PRIX

**En tant que client,**
**Je souhaite bénéficier de réductions**
**De sorte à payer moins cher**

Acceptance

- Prix standard
- Happy Hour (-20%)
- Code promo
- Fidélité (-10%)
- Facile d'ajouter une nouvelle stratégie

wom

# Story 3

64

**VALIDATION DE COMMANDE**

**En tant que système,**
**Je veux valider les commandes**
**Afin de refuser les invalides**

Je connais un pattern qu’on a vu en fin de cours pour ça

Acceptance

- Panier non vide
- Montant minimum 15€
- Zone de livraison OK
- Enseigne ouverte
- Stock disponible
- Règles chaînées, première erreur = stop

wom

# Story 4

65

PAIEMENT

**En tant que client,**
**Je veux payer avec ma méthode préférée**
**Afin de finaliser ma commande**

Acceptance

- 3 moyens de paiement simulés (classes avec `console.log`)
- Chaque moyen a sa propre logique interne (signature différente)

Ici, pas besoin de vraies implémentations. Je veux juste voir trois classes différentes et qu'on puisse passer dynamiquement de l'une à l'autre en cas de problème sur notre plateforme. (Cc Hexa)

wom

# Story 5

66

NOTIFICATIONS

**En tant que client,**
**Je veux  être notifié de l'avancement**
**Afin de suivre ma commande**

Acceptance

- Quand le statut de la commande change, plusieurs "listeners" réagissent
- Chaque listener fait un `console.log` différent (simule email, SMS, etc.)
- Ajouter un listener ne modifie pas la classe `Order`
- On peut activer/désactiver des listeners dynamiquement

# Story 6

67

## ANNULER LES MODIFICATIONS

**En tant que client,**
**Je veux  annuler mes dernières modifications du panier**
**Afin de revenir à un état précédent**

Acceptance

- Chaque modification du panier (ajout, suppression, changement de taille) est sauvegardée
- Le client peut annuler et revenir en arrière (undo)
- Le client peut rétablir une annulation (redo)
- Historique des 10 dernières modifications max

wom

Comment sauvegarder l’état d’un objet pour pouvoir le restaurer plus tard, sans exposer ses détails internes ?
Nous n’avons pas vu ce design pattern en cours. À vous de trouver lequel permet de répondre à ce problème et de justifier votre choix ! 🧐

# Story 7

**69**

**TESTS**

**Tests sur les stratégies de prix**

1 test par stratégie de prix :
- Standard : prix normal
- Happy Hour : -20%
- Code Promo : réduction appliquée
- Fidélité : -10%

Chaque test vérifie le calcul sur une commande type

wom

# Story 8

70

PIZZA PERSONNALISÉE

**En tant que client,**
**Je veux  composer ma propre pizza**
**Afin de choisir exactement ce que je veux**

Acceptance

- Choisir la base : `tomato` ou `cream`
- Choisir la taille : `S`, `M`, `L`, `XL`
- Ajouter des ingrédients à la carte

wom

71

# PIZZA PERSONNALISÉE

**Ingrédients disponibles**

**Mozzarella = 1.50€**
**Pepperoni = 2€**
**Champignons = 1€**
**Jambon = 2€**
**Olives = 1€**
**Gorgonzola = 2.50€**

**Calcul du prix :**
**- Base : 5€**
**- + prix de chaque ingrédient**
**- × multiplicateur de taille (selon l'enseigne)**

# Story 9

72

HISTORIQUE

**En tant que client,**
**Je veux  voir mes commandes passées**
**Afin de les reconsulter ou recommander**

Acceptance

- Liste des commandes du client
- Tri par date (plus récente en premier)
- Filtrage par statut (`delivered`, `cancelled`, etc.)

wom